



ANÁLISIS COMPLETO: seed-complete.ts

Script de Población de Datos para CRTLPyme



RESUMEN EJECUTIVO

El archivo `seed-complete.ts` es un **script completo de población de datos** diseñado para poblar la base de datos PostgreSQL de CRTLPyme con datos realistas de demostración. Este script crea un ecosistema completo de negocios PyME chilenos con histórico de ventas, inventario, usuarios y suscripciones.



HALLAZGO CRÍTICO

El script requiere un archivo externo que NO existe actualmente:

- **Archivo requerido:** `/home/ubuntu/productos_chile.json`
 - **Estado:** **✗ NO ENCONTRADO**
 - **Impacto:** Sin este archivo, el script **FALLARÁ** en la fase de importación de productos
-



TABLAS Y DATOS QUE SERÁN POBLADOS



SubscriptionPlan (Planes de Suscripción)

- **Cantidad:** 3 planes
 - **Datos creados:**
```\n
    - Plan Básico (Mensual)
      - Precio: \$9,990 CLP
      - 2 usuarios, 500 productos máximo
      - 15 días de prueba
    - Plan Pro (Mensual)
      - Precio: \$19,990 CLP
      - 5 usuarios, productos ilimitados
      - 15 días de prueba
    - Plan Enterprise (Mensual)
      - Precio: \$49,990 CLP
      - Usuarios y productos ilimitados
      - 30 días de prueba\n```\n
  - **Comportamiento:** Si ya existen planes, los omite (no duplica)
-

## 2 MasterProduct (Catálogo Maestro de Productos)

- **Cantidad proyectada:** 500 productos
  - **Fuente:** Archivo JSON externo `/home/ubuntu/productos_chile.json`
  - **Campos poblados:**
    - SKU único (auto-generado si no tiene EAN-13)
    - Código de barras (EAN-13)
    - Nombre del producto
    - Categoría
    - Marca
    - Precio sugerido
    - Unidad de medida
  - **Estrategia:** Usa `upsert` (actualiza si existe, crea si no existe)
  - **⚠️ DEPENDENCIA CRÍTICA:** Requiere archivo `productos_chile.json`
- 

## 3 Tenant (Negocios PyME)

- **Cantidad:** 13 negocios chilenos
  - **Tipos de negocios:**
    - Minimarkets (4)
    - Almacenes (4)
    - Supermercados (5)
  - **Distribución de planes:**
    - Plan BASIC: 7 negocios
    - Plan PRO: 5 negocios
    - Plan ENTERPRISE: 1 negocio
  - **Ubicaciones:** Diversas comunas de Santiago
  - **Datos incluidos:**
    - Nombre comercial
    - RUT chileno
    - Email de contacto
    - Teléfono
    - Dirección física
    - Estado: ACTIVE
    - Onboarding completado: true
  - **Comportamiento:** Si el negocio ya existe (por RUT), lo omite
  - **Tenant especial:** Crea "CRTLPyme - Plataforma" (RUT: 99.999.999-9) para el administrador
- 

## 4 User (Usuarios)

- **Cantidad estimada:** 27-40 usuarios
- 1 Administrador de Plataforma (PROVEEDOR)
- 13 Administradores de negocio (ADMIN)
- 13-26 Usuarios operativos (CAJA/INVENTARIO)

- **Distribución por negocio:** 2-3 usuarios por negocio
  - **Roles:**
    - PROVEEDOR : Super Admin de plataforma
    - ADMIN : Administrador del negocio
    - CAJA : Cajero/vendedor
    - INVENTARIO : Encargado de inventario
  - **Credenciales:**
    - Admin Plataforma: admin@crtlpyme.com / Admin2025!
    - Usuarios de negocios: admin@[dominio-negocio] / Demo2025!
    - Usuarios operativos: [nombre][número]@[dominio-negocio] / Demo2025!
  - **Comportamiento:** Si el usuario ya existe (por email), lo omite
- 

## 5 TenantInventory (Inventario de Productos por Negocio)

- **Cantidad estimada:** 650-2,600 registros
  - **Lógica:**
    - Cada negocio tiene entre 50-200 productos en su inventario
    - Los productos se seleccionan aleatoriamente del catálogo maestro
  - **Datos generados por producto:**
    - Precio de costo: 60% del precio sugerido  $\pm 10\%$  de variación
    - Precio de venta: Precio de costo + margen 30%-60%
    - Stock inicial: 5-100 unidades
    - Stock mínimo: 5-15 unidades
  - **Comportamiento:** Si el producto ya existe en el inventario del tenant, lo omite
- 

## 6 Subscription (Suscripciones)

- **Cantidad:** 13 suscripciones (una por negocio)
  - **Características:**
    - Estado: ACTIVE
    - Fecha de inicio: 1-6 meses atrás
    - Auto-renovación: activada
    - Lifetime value: calculado según meses activos
    - Próxima fecha de facturación: calculada automáticamente
- 

## 7 SubscriptionPayment (Pagos de Suscripción)

- **Cantidad estimada:** 39-78 pagos históricos
- **Lógica:**
  - Se crean pagos mensuales históricos por cada suscripción
  - Número de pagos = meses desde el inicio de la suscripción
- **Datos:**
  - Estado: APPROVED

- Método de pago: CREDIT
  - Tarjeta: Visa/Mastercard (aleatorio)
  - Monto: según plan contratado
- 

## 8 Sale (Ventas)

- **Cantidad estimada:** 650-3,900 ventas
  - **Lógica:**
    - Cada negocio genera entre 50-300 ventas
    - Distribuidas en los últimos 90 días (3 meses)
    - Fechas y horarios aleatorios realistas
  - **Datos por venta:**
    - Número de venta secuencial
    - 1-8 productos por venta
    - Subtotal, IVA (19%), total
  - Método de pago (distribución realista):
    - 50% Efectivo
    - 30% Débito
    - 15% Crédito
    - 5% Transferencia
  - Estado: COMPLETED
- 

## 9 SaleItem (Ítems de Venta)

- **Cantidad estimada:** 1,950-27,300 ítems
  - **Lógica:**
    - Promedio 3-4 productos por venta
    - Cantidad: 1-5 unidades por producto
  - **Datos:**
    - Precio unitario
    - Costo unitario
    - Cantidad
    - Subtotal
  - **Impacto:** Actualiza el stock del inventario (resta unidades vendidas)
- 

## 10 StockAdjustment (Movimientos de Inventario)

- **Cantidad estimada:** 65-195 ajustes
- **Lógica:**
  - Cada negocio genera 5-15 movimientos
  - Distribuidos en los últimos 90 días
- **Tipos de movimientos:**
  - **PURCHASE** : Compras a proveedores (10-50 unidades)

- **LOSS** : Pérdidas/mermas (1-5 unidades negativas)
- **CORRECTION** : Correcciones de inventario ( $\pm 5$  unidades)
- **RETURN** : Devoluciones de clientes (1-5 unidades)
- **Impacto**: Actualiza el stock en tiempo real



## RESUMEN CUANTITATIVO

| Tabla               | Registros Mínimos | Registros Máximos | Promedio Estimado |
|---------------------|-------------------|-------------------|-------------------|
| SubscriptionPlan    | 3                 | 3                 | 3                 |
| MasterProduct       | 500               | 500               | 500               |
| Tenant              | 14                | 14                | 14                |
| User                | 27                | 40                | 33                |
| TenantInventory     | 650               | 2,600             | 1,625             |
| Subscription        | 13                | 13                | 13                |
| SubscriptionPayment | 39                | 78                | 58                |
| Sale                | 650               | 3,900             | 2,275             |
| SaleItem            | 1,950             | 27,300            | 14,625            |
| StockAdjustment     | 65                | 195               | 130               |
| <b>TOTAL</b>        | <b>3,911</b>      | <b>34,643</b>     | <b>19,276</b>     |



## DEPENDENCIAS Y PREREQUISITOS



### Dependencias Internas (Incluidas)

1. **Prisma Client**: Incluido en el proyecto
2. **bcryptjs**: Para hash de contraseñas
3. **Node.js fs/path**: Módulos nativos



### Dependencias Externas (FALTANTES)

1. **productos\_chile.json**
  - **Ubicación esperada**: `/home/ubuntu/productos_chile.json`
  - **Estado**: **NO ENCONTRADO**
  - **Estructura esperada**:

```

json
[

```

```
{
 "ean13": "7804123456789",
 "nombre": "Coca Cola 1.5L",
 "categoria": "Bebidas",
 "marca": "Coca Cola",
 "precio_compra": 800,
 "precio_venta": 1200
}
```

- **Impacto: BLOQUEANTE** - Sin este archivo, la población fallará



## Prerequisitos del Sistema

### 1. Base de datos PostgreSQL:

- Debe estar corriendo y accesible
- Variable `DATABASE_URL` configurada
- Migraciones aplicadas ( `prisma migrate deploy` )

### 2. Esquema Prisma:

- Debe estar sincronizado con el código
- Todas las tablas deben existir

### 3. Espacio en disco:

- Estimado: 50-200 MB para los datos generados



## RIESGOS Y CONSIDERACIONES



### Riesgos Críticos

#### 1. ARCHIVO FALTANTE

- `productos_chile.json` no existe
- **Impacto:** El script fallará al intentar importar productos
- **Solución:** Crear el archivo o modificar el script para generar productos sintéticos

#### 2. LIMPIEZA DE BASE DE DATOS

- El script acepta flag `--clean` para borrar TODOS los datos
- **EXTREMADAMENTE PELIGROSO** si se ejecuta en producción
- **Recomendación:** NUNCA usar `--clean` en producción

#### 3. DUPLICACIÓN DE DATOS

- El script usa `upsert` y validaciones de existencia
- Algunos datos se omitirán si ya existen
- **Riesgo:** Inconsistencias si se ejecuta múltiples veces



### Riesgos Medios

#### 1. TIEMPO DE EJECUCIÓN

- Con 13 negocios y miles de registros, puede tardar **5-15 minutos**
- Genera miles de queries a la base de datos
- **Recomendación:** Ejecutar en horario de bajo tráfico

#### 2. STOCK NEGATIVO

- Las ventas históricas restan del inventario

- Si el stock inicial es bajo, podría quedar en negativo
- **Mitigación:** El script genera stock inicial suficiente (5-100 unidades)

### 3. SECUENCIA DE NÚMEROS DE VENTA

- Usa el último número de venta existente
- Si hay ventas previas, continuará la secuencia
- **Riesgo:** Posibles saltos en la numeración

## Consideraciones

### 1. CONTRASEÑAS PREDECIBLES

- Todas las contraseñas son conocidas ( Admin2025! , Demo2025! )
- **Impacto:** Solo para entornos de desarrollo/prueba
- **Acción requerida:** Cambiar contraseñas en producción

### 2. DATOS REALISTAS PERO SINTÉTICOS

- Los negocios son ficticios pero verosímiles
- Los RUTs no son válidos (formato correcto pero no reales)
- Las direcciones son aproximadas

### 3. TRANSACCIONES Y ATOMICIDAD

- El script NO usa transacciones explícitas
- Si falla a mitad de camino, dejará datos parciales
- **Recomendación:** Tener backup antes de ejecutar

## PLAN DE EJECUCIÓN PROPUESTO

### Fase 1: PREPARACIÓN (15-30 minutos)

#### 1.1 Verificación de Pre-requisitos

```
Verificar conexión a base de datos
cd /home/ubuntu/CRTLPyme
npx prisma db pull

Verificar que las migraciones estén aplicadas
npx prisma migrate status

Verificar dependencias Node.js
npm install
```

#### 1.2 Crear Archivo de Productos

##### OPCIÓN A: Generar productos sintéticos (Recomendado)

```
Crear script para generar productos_chile.json
(El asistente puede crear este archivo)
```

##### OPCIÓN B: Usar productos reales

- Solicitar al usuario un archivo CSV/JSON de productos reales
- Convertir al formato esperado

## 1.3 Backup de Seguridad

```
Crear backup de la base de datos actual
pg_dump $DATABASE_URL > /home/ubuntu/backup-pre-seed-$(date +%Y%m%d_%H%M%S).sql
```

---

## Fase 2: REVISIÓN DE DATOS ACTUALES (5 minutos)

### 2.1 Verificar Datos Existentes

```
Ejecutar consultas para verificar estado actual
cd /home/ubuntu/CRTLPyme
npx prisma studio &
Revisar manualmente en http://localhost:5555
```

### 2.2 Consultas SQL de Verificación

```
-- Ver cuántos registros hay actualmente
SELECT
 'Tenants' as tabla, COUNT(*) as registros FROM tenants
UNION ALL
SELECT 'Users', COUNT(*) FROM users
UNION ALL
SELECT 'MasterProducts', COUNT(*) FROM master_products
UNION ALL
SELECT 'TenantInventory', COUNT(*) FROM tenant_inventory
UNION ALL
SELECT 'Sales', COUNT(*) FROM sales
UNION ALL
SELECT 'Subscriptions', COUNT(*) FROM subscriptions;
```

---

## Fase 3: EJECUCIÓN DEL SEED (10-20 minutos)

### 3.1 Ejecución SIN Limpieza (Recomendado para primera vez)

```
cd /home/ubuntu/CRTLPyme
npx ts-node prisma/seed-complete.ts
```

### 3.2 Ejecución CON Limpieza (⚠ PELIGROSO - Solo si es necesario)

```
cd /home/ubuntu/CRTLPyme
npx ts-node prisma/seed-complete.ts --clean
```

### 3.3 Monitoreo de Progreso

- El script imprime mensajes de progreso en consola
  - Observar cada fase completada
  - Verificar que no haya errores
-



## Fase 4: VERIFICACIÓN POST-EJECUCIÓN (10 minutos)

### 4.1 Verificar Conteo de Registros

```
-- Ejecutar nuevamente las consultas de conteo
-- Comparar con los valores esperados
```

### 4.2 Verificar Integridad de Datos

```
-- Verificar que no haya stocks negativos
SELECT * FROM tenant_inventory WHERE stock < 0;

-- Verificar que todas las ventas tengan items
SELECT s.id, s.saleNumber, COUNT(si.id) as items
FROM sales s
LEFT JOIN sale_items si ON s.id = si.saleId
GROUP BY s.id, s.saleNumber
HAVING COUNT(si.id) = 0;

-- Verificar que todos los tenants tengan suscripción
SELECT t.id, t.businessName, s.id as subscriptionId
FROM tenants t
LEFT JOIN subscriptions s ON t.id = s.tenantId
WHERE s.id IS NULL AND t.rut != '99.999.999-9';
```

### 4.3 Probar Credenciales

```
Intentar login con:
- admin@crtlpyme.com / Admin2025!
- admin@minimarketdonluis.cl / Demo2025!
```

### 4.4 Verificar Landing Page

- Acceder a la página de pricing/planes
- Verificar que los 3 planes aparezcan correctamente
- Verificar precios y características

## Fase 5: POST-EJECUCIÓN (5 minutos)

### 5.1 Documentar Resultados

- Capturar estadísticas finales
- Documentar cualquier error o warning
- Anotar contraseñas y usuarios de prueba

### 5.2 Configuración Adicional

- Verificar que las variables de entorno estén correctas
- Revisar configuración de Transbank (modo sandbox)
- Verificar que no se requiera SendGrid para pruebas



## TIEMPO ESTIMADO TOTAL

| Fase              | Tiempo Estimado      |
|-------------------|----------------------|
| 1. Preparación    | 15-30 minutos        |
| 2. Revisión       | 5 minutos            |
| 3. Ejecución      | 10-20 minutos        |
| 4. Verificación   | 10 minutos           |
| 5. Post-Ejecución | 5 minutos            |
| <b>TOTAL</b>      | <b>45-70 minutos</b> |



## RECOMENDACIONES FINALES



### HACER

- ✓ Crear el archivo `productos_chile.json` **ANTES** de ejecutar
- ✓ Hacer backup de la base de datos
- ✓ Ejecutar PRIMERO sin el flag `--clean`
- ✓ Verificar que el entorno sea desarrollo/staging, **NO** producción
- ✓ Revisar las credenciales generadas y documentarlas
- ✓ Verificar la integridad de los datos después de la ejecución



### NO HACER

- ✗ NO ejecutar con `--clean` en producción
- ✗ NO ejecutar múltiples veces sin verificar duplicados
- ✗ NO usar las contraseñas de prueba en producción
- ✗ NO asumir que el archivo de productos existe
- ✗ NO ejecutar sin tener backup
- ✗ NO ignorar los mensajes de error del script



## SOLUCIÓN AL PROBLEMA ACTUAL

Basado en las imágenes proporcionadas, el problema es:

✗ “No hay planes disponibles en este momento”

### Causas Posibles:

- Planes no creados en la base de datos
  - La tabla `subscription_plans` está vacía
  - Solución: Ejecutar el seed

## 2. Planes creados pero no visibles

- Los planes tienen `isVisible: false` o `isActive: false`
- Solución: Verificar y actualizar los registros

## 3. Error en la consulta del frontend

- El componente no está cargando correctamente los planes
- Solución: Verificar logs del servidor

## Plan de Acción Inmediato:

```
1. Verificar si existen planes
cd /home/ubuntu/CRTLPyme
npm run prisma studio
Abrir tabla subscription_plans y verificar

2. Si no hay planes, ejecutar solo la creación de planes:
(Crear script específico para esto)

3. Verificar en el frontend
Acceder a la página de planes y refrescar
```

## PRÓXIMOS PASOS SUGERIDOS

### 1. DECISIÓN DEL USUARIO:

- ¿Desea que se cree el archivo `productos_chile.json` automáticamente?
- ¿Desea ejecutar solo la creación de planes primero?
- ¿Desea ejecutar el seed completo?

### 2. VALIDACIÓN:

- Confirmar que el entorno es desarrollo/staging
- Confirmar que se ha hecho backup
- Confirmar que se entienden los riesgos

### 3. EJECUCIÓN:

- Proceder según lo acordado
- Monitorear en tiempo real
- Documentar resultados

## RESUMEN DE CREDENCIALES QUE SE CREARÁN

### Administrador de Plataforma

Email: `admin@crtlpyme.com`  
 Contraseña: Admin2025!  
 Rol: PROVEEDOR (Super Admin)  
 Tenant: CRTLPyme - Plataforma

## Ejemplo de Usuarios de Negocios

```
Minimarket Don Luis
Email: admin@minimarketdonluis.cl
Contraseña: Demo2025!
Rol: ADMIN

Almacén El Rinconcito
Email: admin@gmail.com (del dominio del negocio)
Contraseña: Demo2025!
Rol: ADMIN
```

## Usuarios Operativos

```
Patrón: [nombre][número]@[dominio-negocio]
Ejemplo: juan1@minimarketdonluis.cl
Contraseña: Demo2025!
Rol: CAJA o INVENTARIO
```

---

## ADVERTENCIA FINAL

Este script está diseñado para **entornos de desarrollo y demostración**. Los datos generados son realistas pero sintéticos. **NO debe ejecutarse en un entorno de producción con datos reales** sin una revisión exhaustiva y adaptación a los requisitos específicos del negocio.

---

**Generado:** 2025-11-12

**Versión del Script:** seed-complete.ts (commit ebfb66cc)

**Estado del Análisis:** COMPLETO 